

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/266214504>

# Extraction of cylinders and estimation of their parameters from point clouds

Article in *Computers & Graphics* · February 2015

DOI: 10.1016/j.cag.2014.09.027

CITATIONS

40

READS

2,796

3 authors:



**Trung-Thien Tran**

18 PUBLICATIONS 105 CITATIONS

[SEE PROFILE](#)



**Van-Toan Cao**

Laval University

15 PUBLICATIONS 86 CITATIONS

[SEE PROFILE](#)



**Denis Laurendeau**

Laval University

241 PUBLICATIONS 2,839 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Robotic in brachytherapy [View project](#)



Computer vision, computer graphics [View project](#)



SMI 2014

# Extraction of cylinders and estimation of their parameters from point clouds

Q1 Trung-Thien Tran, Van-Toan Cao, Denis Laurendeau

Q2 Laval University, 1065, avenue de la Medecine, Quebec, Canada G1V 0A6

## ARTICLE INFO

### Article history:

Received 4 July 2014

Received in revised form

23 September 2014

Accepted 27 September 2014

### Keywords:

Cylinder extraction

Cylinder fitting

Point cloud

RANSAC

Mean shift clustering

## ABSTRACT

In this paper, a novel algorithm is proposed for extracting cylinders and estimating their parameters from 3D point cloud data. First, normal vectors and curvature information are computed for each data point as a preprocessing step. Then potential points that could belong to cylindrical surfaces are extracted by using curvature information. For each potential cylinder point, its neighborhood is considered as inlier points and a robust cylinder fitting algorithm is applied on these inlier points. Inlier points are updated and the fitting process is applied iteratively to propagate to all remaining points belonging to a cylinder. A validation method is proposed to assess whether the detected cylinder is reliable or not. Finally, by applying mean shift clustering, final descriptive parameters of cylinders are estimated accurately. To demonstrate its robustness, the method is tested on both synthetic and complex point clouds with different levels of noise and outliers.

© 2014 Elsevier Ltd. All rights reserved.

## 1. Introduction

Due to the rapid development of scanning technologies, very large sets of accurate 3D points can be collected easily and quickly. Therefore, more and more computer vision and graphics applications have used such scanners, especially in industrial manufacturing. Cylinder extraction and cylinder fitting are the problems that have been investigated recently, but which are still open.

Most manufactured objects are composed of basic primitives such as planes, spheres, cylinders, toruses and cones [1,2]. Cylinders are the most frequently used primitive in these objects such as pipes, sleeves, connectors. Moreover, cylinder detection could be used in various applications: reverse engineering [3], 3D registration [4], pipeline plant modeling [5], etc. Currently, there are many methods [5–8] that have investigated how to detect cylinders and estimate their descriptive parameters from point cloud data. A cylinder is frequently described by 3 parameters: axis orientation  $\vec{a}$ , a point on the axis  $p^*$  and radius  $r$  [1]. However, few methods can determine these descriptive parameters accurately. Most methods [2,5–7] require the user to choose some threshold values—a difficult task for complex models. Moreover, the problem of detecting multiple cylinders is non-trivial, since it is easily affected by noise/outliers or depends on prior segmentation results.

Therefore, our goal is to propose a robust method for estimating cylinder parameters accurately and extracting multiple cylinders simultaneously from a given point cloud. Some examples of cylinder extraction detected by our method are shown in Fig. 1.

<http://dx.doi.org/10.1016/j.cag.2014.09.027>

0097-8493/© 2014 Elsevier Ltd. All rights reserved.

The contributions of the proposed method are summarized as follows: (i) estimating the descriptive parameters of cylinders accurately; (ii) extracting multiple cylinders at the same time; (iii) proposing a novel method for validating the detected cylinders; (iv) detection and estimation robust to acquisition of noise and outliers.

The rest of this paper is organized as follows. Previous work is summarized in Section 2. Single cylinder fitting is presented in Section 3. The procedure of multiple cylinder extraction is described in Section 4. Results and discussion are presented in Section 5, and Section 6 draws some conclusions on the proposed method.

## 2. Related work

Many methods have investigated the problem of cylinder extraction. In this section, we briefly review some methods related to the one presented in this paper. Existing methods are generally classified into two categories: those requiring prior segmentation and those working directly on point clouds.

The methods belonging to the first group fit a cylinder model to a set of segmented points. They often use non-linear optimization to minimize the sum of orthogonal distances to the cylinder surface [9,10]. These methods depend on the quality of the initial segmentation [11] that needs to assign the correct type of primitive to each data point. However, this requirement is difficult to meet for real data with noise and outliers. Moreover, non-linear

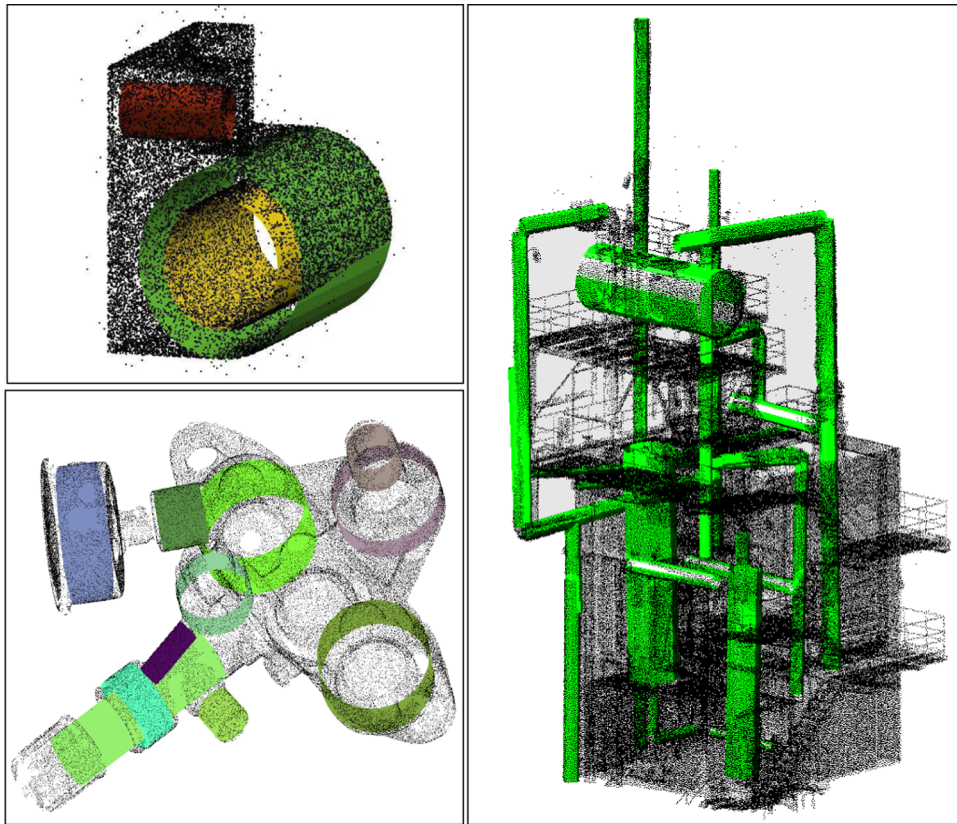


Fig. 1. Detected cylinders from noisy data with outliers. Zoom in to view noisy data and outliers.

least-squares fitting is an iterative process that requires good initial parameters to avoid local minima.

Hierarchical clustering [12,1], which often requires that the number of segments be pre-selected by the user, is a good method for primitive extraction. However, this method is computationally intensive in time complexity and memory even if the model contains just a few thousand points, because the method investigates all types of primitives at the same time to create a global list of priorities by using algebraic fitting [13]. Moreover, primitive fitting is not accurate, especially when the number of points is small. This affects the priority list leading to poor results. In addition, cylinder fitting in [12,1] cannot identify cylinder parameters accurately and reliably in a single step. Fig. 3(a) shows an example of this method after one iteration step, the detected cylinder does not fit the data points very well. Our method improves this technique to estimate cylinder parameters accurately through an iterative fitting process.

The second group attempts to process data points directly using RANSAC [14,2] or Hough-based methods [7,6] which are popular techniques for parametric model extraction. RANSAC [15,16] fits a model by using random sampling with minimum number of data points. For cylinders, two data points with their surface normals are enough for fitting [2]. However, the user must set several thresholds that may vary for different models, especially with noisy data and outliers. The results of the algorithm depend on the initial selection of points. In the worst case, RANSAC does not converge to a valid solution even when cylinders are present.

Hough transform [17] is popular for detecting simple shapes such as lines, circles and planes. But it is also known as a voting method with high computational requirements, and for which choosing the resolution of the accumulator cell is critical. In addition, 5 parameters need to be used for cylinder detection [7]. As reported in [7,18], some

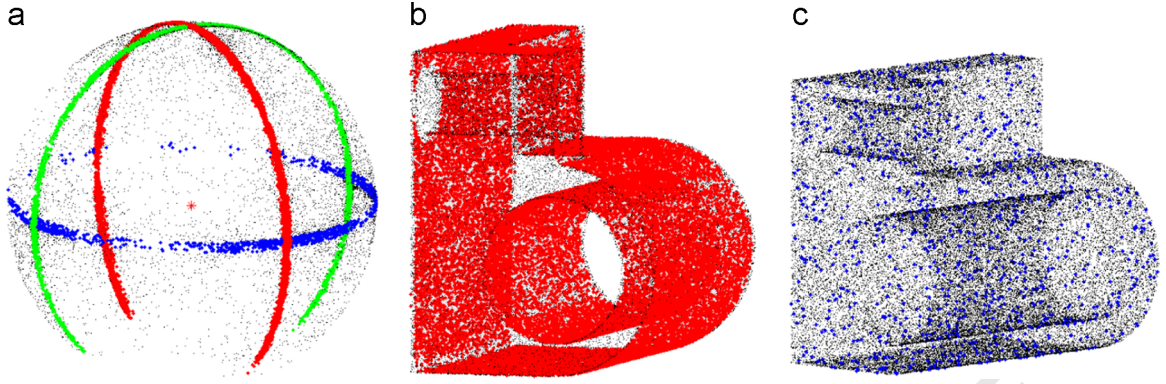
authors have attempted to speed up the procedure by splitting and pruning the parametric space.

A significant research study focuses on the use of the Gaussian sphere [19] to extract primitives [5–7]. Liu et al. [5] detect cylinders in large-scale point clouds of a pipeline plant. However, the limitation of the method is that it only supports the extraction of pipes either perpendicular or parallel to the ground. Chaperon et al. [6] combine the Gaussian sphere with a random sampling method (RANSAC) to extract cylinders and estimate their parameters. However, based on our experiments, these methods do not achieve accurate results for complex models containing other types of primitives. For example, in Fig. 2(a), the joint model is made of 3 cylinders and several planes leading to red and green circles, where two parallel cylinders correspond to the same red circle. However, the red circle results not only from points belonging to cylinders but also outliers and other points on a plane having coincident normals (Fig. 2(b)). The blue circle is not created by the points belonging to the cylinder surface but rather by points distributed all over the model (Fig. 2(c)).

The proposed method belongs to the second group, which extracts cylinders directly from point cloud data. The problem addressed in this paper is to estimate cylinder parameters accurately. The algorithm is an iterative process of distance-based and normal-based fitting starting at different initial points. Furthermore, we propose a novel two-step validation method exploiting geometric consistency to improve cylinder detection. By using mean shift clustering, multiple cylinders are extracted simultaneously.

### 3. Iterative cylinder fitting

In this section, we assume that points are sampled on the surface of a cylinder, which includes noise and outliers. The case of



**Fig. 2.** Gaussian sphere of joint model in top left of Fig. 1 and related problems. The blue circle is caused by planes and outliers and leads to wrong results. The problem of detecting circles on the Gaussian sphere becomes harder when there are many cylinders or when there are small cylinders in the point clouds. (a) Gaussian sphere of joint model contains three circles. (b) Red circle is created by the red points belonging to cylinders and planes. (c) Blue circle is created by the blue points distributed all over the model. (For interpretation of the references to color in this figure caption, the reader is referred to the web version of this paper.)

multiple cylinder detection with the presence of other types of primitives is investigated in the next section. The pseudo-code for single cylinder fitting is shown in Algorithm 1. The steps of this method are described in more detail as follows.

**Algorithm 1.** Algorithm for iterative single cylinder fitting.

**Data:** Point cloud and normal vectors at each point.

**Result:** Parameters  $\{p^*, \vec{a}, r\}$  of fitted cylinder.

index:=0;  $k:=10$  (number of iterations);

inlier:=a given point and its neighborhood whose size is initially set to 50;

**while** index  $\leq k$  **do**

1. Find the cylinder axis orientation  $\vec{a}$  by finding the eigenvector corresponding to the smallest eigenvalue of the covariance matrix  $C$  of normal vectors of inliers;
2. Project inlier on a plane with normal  $\vec{a}$  and going through the origin  $O$ ;
3. Fit a circle to projected points to find radius  $r$  and a point  $p^*$  belonging to the axis (Eq. (1)–(2));
4. Use normal and distance filtering to update inlier points for the next iteration (Eq.(3));
5. index:=index+1;

**end**

The input of the algorithm is a set of points sampled on the surface of a cylinder  $P = \{p_i \in \mathbb{R}^3, i = 1, \dots, N\}$ . An un-oriented normal vector is available at each point. Otherwise, normal vectors can be computed using the procedure described in [20]. For the sake of completeness, normal computation is explained in more detail in Appendix A. The output of the method is a set of parameters describing the detected cylinder  $\{p^*, \vec{a}, r\}$ .

A cylinder is described by a set of 3 parameters: axis orientation vector  $\vec{a}$ , a point on the axis  $p^*$  and radius  $r$ . The first step is to compute the orientation of the axis of the cylinder, that is the vector being the most orthogonal to all the normal vectors  $\vec{n}_i$  of inliers (see Eq. (3)). Therefore, a positive semi-definite matrix  $C = \sum_{i=1}^{n_{\text{inlier}}} \vec{n}_i \vec{n}_i^T$  is computed and analyzed, in which  $\vec{n}_i$  is a row-vector (row-vector convention is used in this paper). The eigenvector of  $C$  corresponding to the smallest eigenvalue is considered as the axis orientation  $\vec{a}$ , and  $C_x$  and  $C_y$  are the two other eigenvectors that create a coordinate frame in the plane.

In the second step,  $\vec{p}_i = [\langle p_i, C_x \rangle, \langle p_i, C_y \rangle]$  is the 2D projection of  $p_i \in \text{inlier}$  on the plane with normal  $\vec{a}$  that passes through the

origin  $O$ . These projected points distribute on the plane as a circle. Then the problem of determining the radius  $r$  and a point  $p^*$  on the axis reduces to determining the radius  $r$  and center  $\tilde{c}$  of the circle. Algebraic fitting [13] is used for computing the radius  $r$  and center  $\tilde{c} = [\tilde{c}_x, \tilde{c}_y]$  because it is faster than implicit fitting [11] and efficient for the circle fitting problem. Therefore, algebraic distances are minimized in the following equation:

$$[\tilde{c}_x, \tilde{c}_y, r^2 - \tilde{c}_x^2 - \tilde{c}_y^2]^T = (A^T A)^{-1} A^T b \quad (1)$$

where

$$A = \begin{pmatrix} 2\tilde{p}_{1x} & 2\tilde{p}_{1y} & 1 \\ 2\tilde{p}_{2x} & 2\tilde{p}_{2y} & 1 \\ \vdots & \vdots & \vdots \\ 2\tilde{p}_{n_{\text{inlier}}x} & 2\tilde{p}_{n_{\text{inlier}}y} & 1 \end{pmatrix}, \quad b = \begin{pmatrix} \|\tilde{p}_1\|_2^2 \\ \|\tilde{p}_2\|_2^2 \\ \vdots \\ \|\tilde{p}_{n_{\text{inlier}}}\|_2^2 \end{pmatrix} \quad (2)$$

The center  $\tilde{c}$  of the circle is transformed back in 3D coordinates and considered as the point  $p^*$  belonging to the axis of the cylinder, i.e.  $p^* = \tilde{c}_x C_x + \tilde{c}_y C_y$ .

We iteratively apply these three steps for inlier points. At each iteration, the set of inliers (line 4 in Algorithm 1) is updated with the following normal and distance criterion:

$$\text{inlier} = \left\{ i \mid \left( \left| \Delta \text{dist}_i \right| < \frac{r}{\alpha} \right) \& \left( \left| \Delta \text{normal}_i \right| > \beta \right) \right\} \quad (3)$$

where  $\Delta \text{dist}_i$  is the distance of point  $p_i \in P$  to the detected cylinder and  $\Delta \text{normal}_i$  is the angle between its normal  $\vec{n}_i$  and the vector from point  $p_i$  to the axis, as shown in Fig. 4(b). The terms  $\Delta \text{dist}_i$  and  $\Delta \text{normal}_i$  are computed as follows:

$$\Delta \text{dist}_i = \left\| \overrightarrow{p_i - p^*} - (\overrightarrow{p_i - p^*} \cdot \vec{a}) \vec{a} \right\| - r \quad (4)$$

and

$$\Delta \text{normal}_i = \frac{\cos \left( \frac{(\overrightarrow{p_i - p^*} - (\overrightarrow{p_i - p^*} \cdot \vec{a}) \vec{a}) \cdot \vec{n}_i}{\|\overrightarrow{p_i - p^*} - (\overrightarrow{p_i - p^*} \cdot \vec{a}) \vec{a}\|} \right)}{\|\overrightarrow{p_i - p^*} - (\overrightarrow{p_i - p^*} \cdot \vec{a}) \vec{a}\|} \quad (5)$$

where  $(\cdot)$  is the dot product of two vectors.

Parameters  $\alpha$  and  $\beta$  in Eq. (3) are adaptive threshold values for distance and orientation differences respectively. Large values for  $r/\alpha$  and  $\beta$  make the process greedy for merging data points into the fitting process, but small ones make the process converge more slowly. Therefore,  $\alpha = 50$  and  $\beta = 0.95$ , where  $r$  is the radius of the detected cylinder, are good values found empirically and were used for all experiments reported in this paper. These threshold values are used to update the set of inliers (see Eq. (3)) automatically at each iteration.

For each iteration, only inlier points are considered as input for cylinder fitting. This process propagates to points belonging to the cylinder surface rapidly, as shown in Fig. 3. The major advantage of this method consists of choosing the inliers iteratively based on distance and normal information, so outliers and noise are sequentially removed by using the distance and normal filter in Eq. (3). Therefore, the descriptive parameters of a cylinder are estimated by inliers only, which makes the estimates accurate.

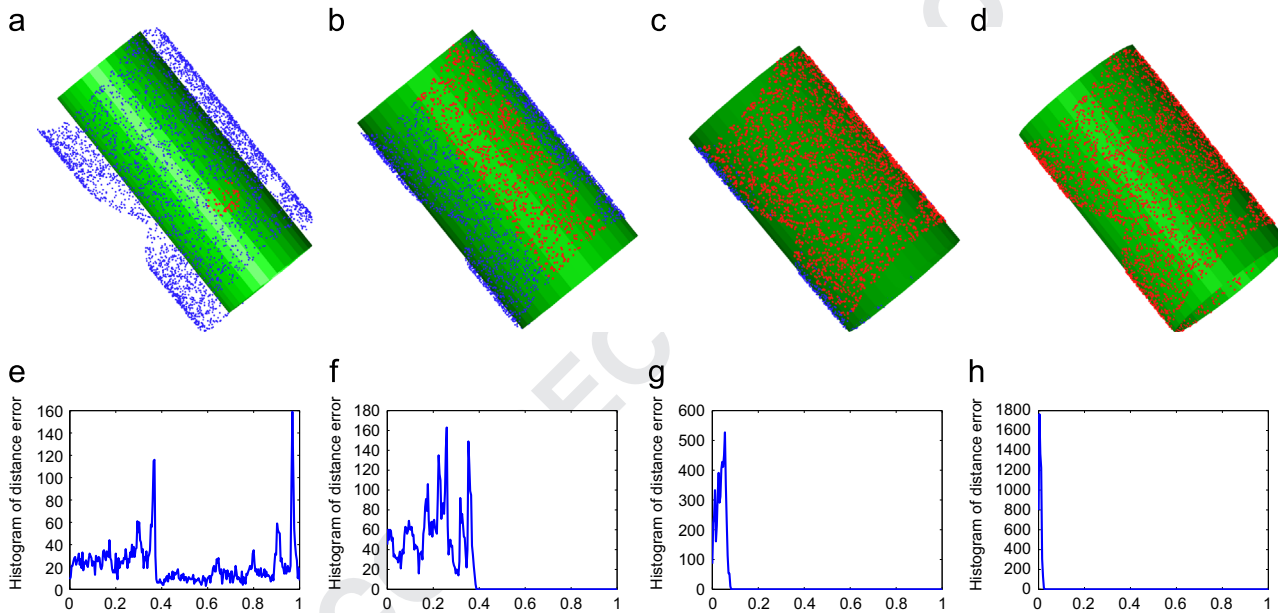
Some practical experiments were conducted to choose the number of iterations  $k$  used in Algorithm 1. We applied the above process to different models of cylinders. The distance  $\Delta dist_i$  between the points and the surface of the detected cylinder was computed after each iteration. The mean square error (MSE) of the distance error was computed as follows:

$$MSE = \frac{1}{N_i \in N} \sum (\Delta dist_i)^2 \quad (6)$$

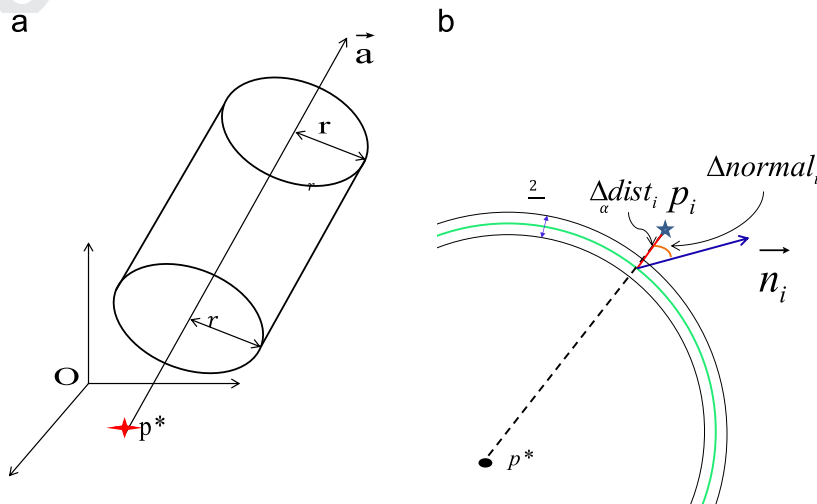
The MSE of different models is normalized and plotted in Fig. 5. For all the models with  $\alpha$  and  $\beta$  mentioned above, MSE converges to a small value after a number of iteration smaller than 10. Therefore,  $k$  was set to 10 in all the experiments.

The proposed method is based on hierarchical clustering (HC) [1], but it outperforms HC's performance on many aspects:

- Hierarchical clustering is similar to region growing and merges just one point to the cluster at each iteration, this makes the process slow because of high computational cost. While the proposed method rather detects and merges multiple points at each iteration, which speeds up the overall process.
- Hierarchical clustering is dependent on a priority list which is affected easily by noisy data because the error fitting for a sphere or a plane is often smaller than that of a cylinder when the number of points is small. The proposed method avoids these problems by focusing on local data continuously and by



**Fig. 3.** Iterative fitting for cylinder extraction. Red points are inliers propagated gradually on the surface of the cylinder. Distance error (Fig. 4(b)) computes the distance from a point to the surface of the detected cylinder. The error gradually drops close to zero after a few iterations as shown on the histograms of distance error. (a) After 1st iteration. (b) After 2nd iteration. (c) After 3rd iteration. (d) After 4th iteration. (e) Histogram of distance error after 1st iteration. (f) Histogram of distance error after 2nd iteration. (g) Histogram of distance error after 3rd iteration. (h) Histogram of distance error after 4th iteration.



**Fig. 4.** Computation of the orthogonal distance  $\Delta dist_i$  (red line) between a point  $p_i$  and the detected cylinder (green curve). Angle  $\Delta normal_i$  (orange) between its normal vector and a vector on the detected cylinder surface.  $p_i$  is the point of interest.  $p^*$  is a point on the axis of the detected cylinder.  $\vec{n}_i$  is the normal vector at a given point  $p_i$ .  $2r/\alpha$  is the margin for inlier points. (a) Cylinder model and its descriptive parameters. (b) Top view of cylinder.

using normal and distance criterion to remove noisy data and outliers.

- As shown in Section 5, the proposed method is faster and more reliable than HC and other methods. The method is able to

diagram of the method is given in Fig. 6 and Algorithm 2. Each block is described in the following.

**Algorithm 2.** Multiple cylinders extraction from point cloud.

**Data:** Point cloud; Normal vectors (Appendix A) and Potential points (Section 4.1)

**Result:** Parameters of cylinders

index:=0; m:=# Potential points; CYLINDER:=∅;

**while** index ≤ m **do**

1. Find neighborhood for a given initial seed point and then fit a cylinder using Section 3, Algorithm 1;

2. Validation by *firstphase* Eq. (7), Section 4.2.1;

3. **If true then**

4. Validation by *second phase* by computing **score** Eq. (9) and Algorithm 3;

5. **if** score <  $\epsilon$  **then**

| CYLINDER = CYLINDER ∪ newcylinder;

**end**

**end**

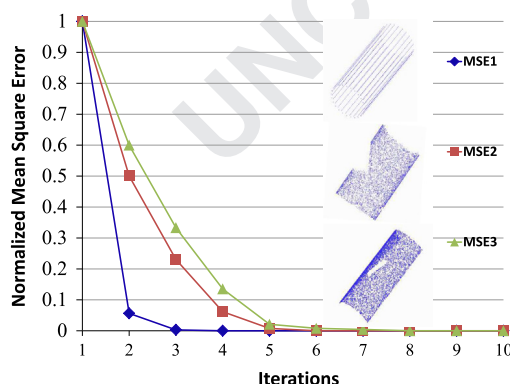
6. index:=index+1;

**end**

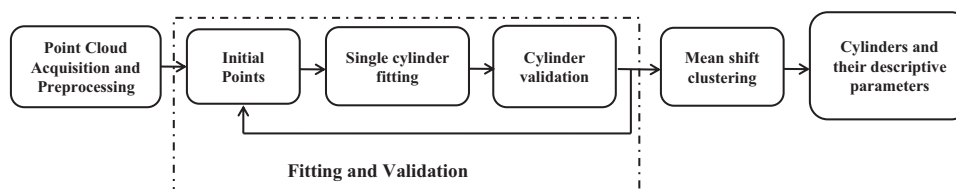
estimate cylinder parameters accurately, even in the presence of noise and outliers. Fig. 3 shows the results of cylinder fitting. After 4 iterations, all points belonging to the cylinder surface are detected with an accuracy.

#### 4. Multiple cylinder extraction

As mentioned before, manufactured objects are generally made of multiple cylinders combined with other primitives. Therefore, choosing only points belonging to cylinder surfaces for fitting is not a trivial problem. In this section, a robust method for the simultaneous detection of multiple cylinders is proposed. The



**Fig. 5.** Iterations for different models: complete synthetic cylinder, cylinder with hole and partial cylinder. Mean Square Error (MSE) of distance error converges to zero after a few iterations smaller than 10.



**Fig. 6.** Diagram for multiple cylinder extraction.

The result of the fitting process depends on the initial seed point and the propagation process. Because of acquisition noise or point cloud structure, some detected cylinders may not converge to a good cylinder and the detection process could be stuck in a geometry trap as shown in Fig. 7(e). To eliminate such cases, a novel method for validating the detected cylinder is proposed. The validation algorithm consists of two phases: (i) a first phase checks the geometric consistency of the detected cylinder; (ii) a second phase checks the local structure around it.

#### 4.2.1. First phase of the validation step

The first phase of the validation step verifies the geometric consistency of the detected cylinder. Based on the experiment reported in Fig. 5, the estimation of the parameters of good cylinders converge after a few iterations only. Therefore, the convergence property is used to verify the geometric consistency of a cylinder. A cylinder primitive is considered as reliable one if it satisfies the following conditions:

$$\begin{cases} \|r_k - r_{k-1}\| < \gamma \\ \frac{(\vec{a}_k \cdot \vec{a}_{k-1})}{(\vec{a}_{k-1} \cdot \vec{a}_{k-2})} > \theta \end{cases} \quad (7)$$

where the number of iterations  $k$  is set to 10 mentioned in Algorithm 1.

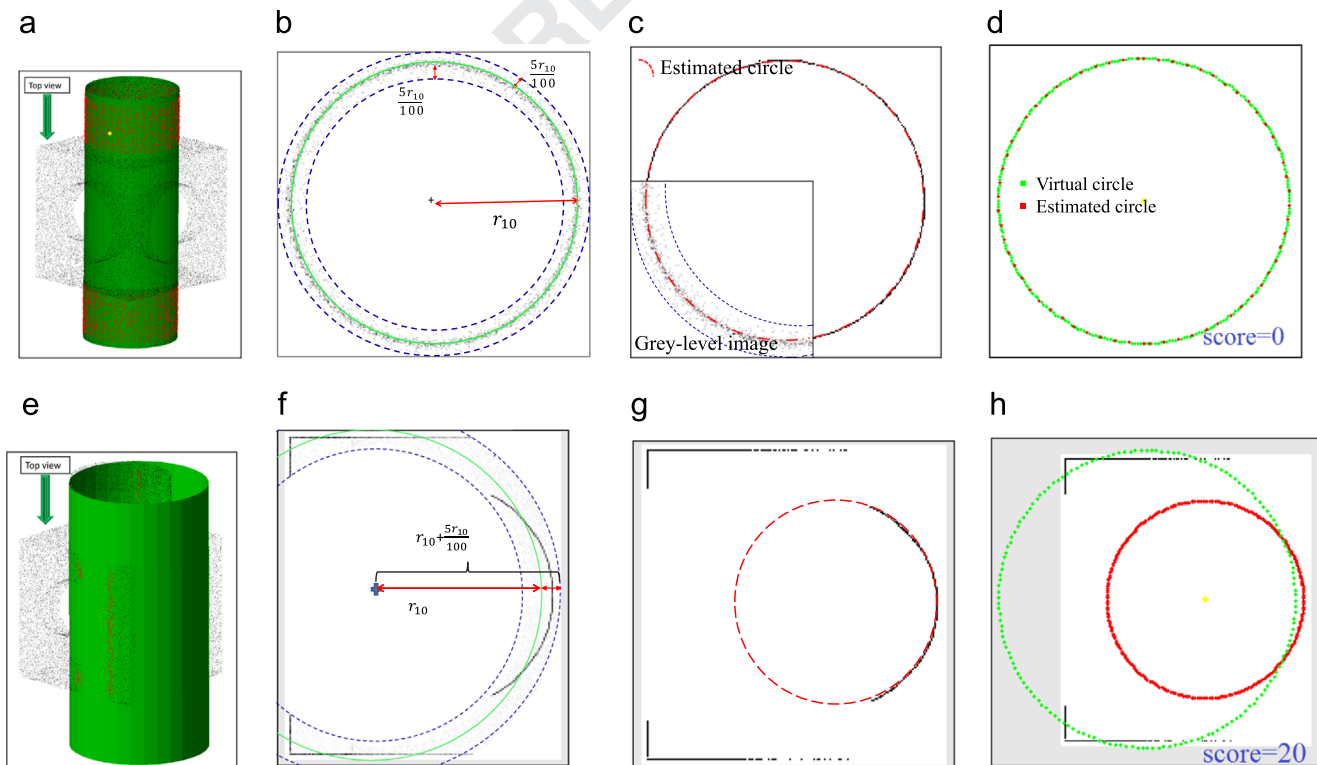
The first inequality evaluates the convergence of the radius  $r$ , and the second one assesses the robustness of axis orientation  $\vec{a}$ . In our experiments  $\gamma = \frac{1}{100} \sum_{i=0}^{10} r_i / 10$  is set as 1% of the average radius of the detected cylinders at all iterations and  $\theta = 0.99$ . After the first phase, only the detected cylinders satisfying Eq. (7) are fed to the second phase for further investigation.

#### 4.2.2. Second phase of the validation step

As the first phase may still lead to unreliable cylinders because the fitting process is trapped in a local minimum in geometry, the second phase is executed. The main idea of this second phase is to explore the structure around the detected cylinder to assess its quality.

- Virtual circle  $\{\tilde{c}_{vir}; r_{vir}\}$  is generated by projecting the detected cylinder on a plane normal to the axis direction  $\vec{a}$  and through origin O.
- All points near the surface of the detected cylinder are also projected on the same plane and generate a circular shape which is detected and called the *estimated circle*  $\{\tilde{c}_{est}; r_{est}\}$ .
- The problem is thus transformed into one of detecting a circle in the plane. The detected cylinder is reliable if these two circles coincide closely (Fig. 7(d)).

The second phase of the validation step is summarized in Algorithm 3 and explained as follows. First, the detected cylinder is projected on a plane, which has normal  $\vec{a}$  and passes through origin O, to generate a circle called the *virtual circle* (green in Fig. 7(b)–(f)). Then the data points within the  $(5 \cdot r_{10})/100$  margin (Fig. 7(b)–(f)) from the surface of the detected cylinder are also projected on the same plane and distributed as a circle. These projected points are stored into a  $5 \times 5$  accumulator-grid that is then normalized to produce a grey-level image (Fig. 7(b)–(f)). The size of the accumulator cell is automatically set to  $r_{10}/100$ , which is chosen to balance between accuracy, complexity and computational efficiency. The resolution of the grey level image is thus



**Fig. 7.** Validation of cylinder detection. With a good cylinder, both the virtual circle and estimated circle coincide. On the other hand, this condition is not satisfied even if the detected cylinder passes the first validation step. Therefore, it is considered as an unreliable cylinder and rejected. (a) Valid detected cylinder. Yellow dot is the initial point. (b) Grey level image of the projection (top view). Dashed blue circles are  $5r_{10}/100$  margins. (c) A part of grey-level image and its binary image of the projection. (d) Green dots are resampled from the virtual circle, red dots are resampled from the estimated circle. (e) Non-valid detected cylinder. (f) Half-size computation of accumulator grid. (g) Detected cylinder creates a virtual circle (green). (h) Dashed red circle is an estimated circle in the binary image. (h) Virtual circle and estimated circle do not coincide which results in a large value of the score.

210 × 210 and is computed as follows (Fig. 7(f)):

$$S = 2 * \frac{r_{10} + 5 * \frac{r_{10}}{100}}{\frac{r_{10}}{100}} = \frac{2 * 100 * \frac{r_{10}}{100} + 2 * 5 * \frac{r_{10}}{100}}{\frac{r_{10}}{100}} = 210 \quad (8)$$

**Algorithm 3.** Second phase of the validation step.

- Data:** Point cloud; the descriptive parameters of the detected cylinder that passed the first phase.
- Result:** **score** assessing the difference between the estimated circle and the virtual circle.
1. Project the detected cylinder on a plane with normal  $\vec{a}$  and passing through origin O and generate the *virtual circle*  $\{\tilde{c}_{vir}; r_{vir}\}$ ;
  2. Project the data points within  $\frac{5 * r_{10}}{100}$  margin from the surface of the detected cylinder on the same plane;
  3. Store and count the projected points by using an accumulator grid with cell size  $\frac{r_{10}}{100}$ ;
  4. Normalize accumulator to create a grey-level image;
  5. Generate a binary image from the grey-level image [25];
  6. Detect a *estimated circle*  $\{\tilde{c}_{est}; r_{est}\}$  using RANSAC;
  7. Compute **score** using Eq. (9);

A binary image is then generated using Otsu's method [25]. The problem is reduced to detecting a circle in this binary image, which is called the *estimated circle* (Fig. 7(c)–(g)). Finally, the virtual circle of the detected cylinder and the estimated circle from binary image are compared. Based on our experiments, RANSAC is a good method for detecting circles in a binary image [26,5]. The difference between the estimated circle and the virtual circle is assessed by a **score** that is the sum of the center deviation and radius deviation in pixels.

$$score = sum \left( \begin{bmatrix} \tilde{c}_{vir} \\ r_{vir} \end{bmatrix} - \begin{bmatrix} \tilde{c}_{est} \\ r_{est} \end{bmatrix} \right) \quad (9)$$

where  $\tilde{c}_{vir}$  and  $r_{vir}$  are the center and radius of the virtual circle respectively, and  $\tilde{c}_{est}$  and  $r_{est}$  are the center and radius of the estimated circle in the binary image (red in Fig. 7(c)–(g)) respectively. Only the cylinders that satisfy the condition ( $score < \epsilon$ ) are considered as reliable primitives (Fig. 7(d)). Otherwise, they are rejected (Fig. 7(h)).  $\epsilon$  is chosen according to the level of noise in the 3D point cloud.

After the two validation phases, reliable cylinders are identified and their descriptive parameters are stored into a **CYLINDER** list.

### 4.3. Mean shift clustering

Once the above steps are executed, descriptive parameters for multiple cylinders are stored into a **CYLINDER** list. Potential points belonging to the same cylinder should have the same set of descriptive parameters. Fig. 8(a) shows the histogram of radius values of detected cylinders. The next step consists of clustering points sharing the same set of parameters and leading to the detection of multiple cylinders. A non-parametric clustering approach, mean shift clustering [27], is used for this purpose. Contrarily to k-means clustering methods, mean shift clustering does not require the number of clusters to be chosen a priori.

*Mean Shift clustering* [27] works on grouping data into a set of clusters: Given  $n_c$  data points  $\chi = \{X_i | i = 1, \dots, n_c\} \subset R^d$  sampled from a density  $f(X)$ . The multivariate kernel density estimator with a symmetric kernel,  $K(X)$ , is given by

$$\hat{f}(X) = \frac{1}{n_c h^d} \sum_{i=1}^{n_c} K\left(\frac{X - X_i}{h}\right) \quad (10)$$

where  $h$  is called the bandwidth parameter. The symmetric kernel is defined as

$$K(X) = qk(\|X\|^2) \quad (11)$$

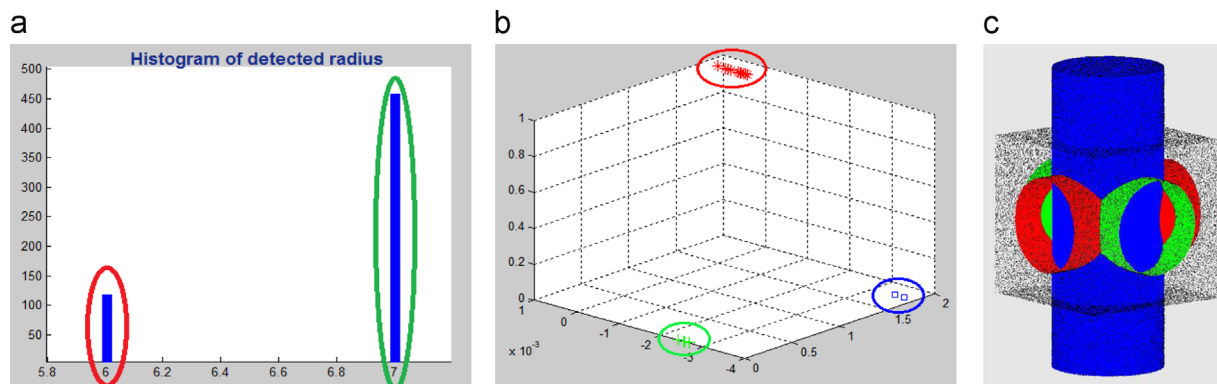
where  $k(X)$  is called the profile function and  $q$  represents a normalizing constant:

$$\int_{R^d} k(\|X\|^2) dX = \frac{1}{q} \quad (12)$$

$\hat{f}(X)$  has local maxima at the cluster centers. The simplest method to find the local maxima is to use a gradient-ascent technique. Taking the gradient of the density estimator in Eq. (10) and performing further algebraic manipulations yields

$$\nabla \hat{f}(X) = \frac{2q}{n_c h^{d+2}} \sum_{i=1}^{n_c} g\left(\left\|\frac{X - X_i}{h}\right\|^2\right) \times \left[ \frac{\sum_{i=1}^{n_c} X_i g\left(\left\|\frac{X - X_i}{h}\right\|^2\right)}{g\left(\left\|\frac{X - X_i}{h}\right\|^2\right)} - X \right] \quad (13)$$

where  $g(X) = -k'(X)$  denotes the derivative of the selected kernel profile. The first term  $g\left(\left\|\frac{X - X_i}{h}\right\|^2\right)$  is proportional to the density estimate at  $X$ . The second term is called the mean shift



**Fig. 8.** Mean shift clustering and final results. (a) In radius space, there are two clusters since the block model has three cylinders, but two having the same radius. (b) There are three clusters in the  $p^*$  space. (c) The final results after mean shift clustering: three cylinders are detected correctly.

vector  $m$ ,

$$m(X) = \frac{\sum_{i=1}^{n_c} X_i g\left(\left\|\frac{X-X_i}{h}\right\|^2\right)}{g\left(\left\|\frac{X-X_i}{h}\right\|^2\right)} - X \quad (14)$$

The mean shift vector points toward the direction of maximum increase in density and is proportional to the density gradient estimate at point  $X$ . Therefore, the mean shift procedure for a given point  $X_i$  is as follows:

1. Compute the mean shift vector  $m(X_i^t)$ .
2. Translate the density estimation window:  $X_i^{t+1} = X_i^t + m(X_i^t)$ .
3. Iterate steps 1 and 2 until convergence.

In this paper, we apply mean shift clustering in cylinder parameters space  $\chi \in R^7$ . Each detected cylinder is considered as a data point  $\chi = \{X_i = (\vec{a}_i, p_i^*, r_i)\}$ . In this problem, mean shift clustering with separable Gaussian kernels is used:

$$\hat{f}(X) = \frac{1}{n_c h_1 h_2 h_3} \sum_{i=1}^{n_c} K_1\left(\frac{\vec{a} - \vec{a}_i}{h_1}\right) K_2\left(\frac{p^* - p_i^*}{h_2}\right) K_3\left(\frac{r - r_i}{h_3}\right) \quad (15)$$

The mean shift vector  $m$  is currently defined as

$$m(X) = \frac{\sum_{i=1}^{n_c} X_i g\left(\left\|\frac{\vec{a} - \vec{a}_i}{h_1}\right\|^2\right) g\left(\left\|\frac{p^* - p_i^*}{h_2}\right\|^2\right) g\left(\left\|\frac{r - r_i}{h_3}\right\|^2\right)}{g\left(\left\|\frac{\vec{a} - \vec{a}_i}{h_1}\right\|^2\right) g\left(\left\|\frac{p^* - p_i^*}{h_2}\right\|^2\right) g\left(\left\|\frac{r - r_i}{h_3}\right\|^2\right)} - X \quad (16)$$

The only parameters that must be defined are bandwidth values  $h = \{h_1, h_2, h_3\}$ .  $h_1$  is the bandwidth for angle difference of axis orientation  $\vec{a}$ ,  $h_2$  is the bandwidth for the points  $p^*$  on the axis, and  $h_3$  is the bandwidth for the radius. In all experiments reported in this paper, the bandwidth is set automatically  $h = \{0.01, \sigma, \frac{1}{100} \sum_{i=1}^{n_c} r_i / n_c\}$  where  $\sigma$  is the average distance between point cloud data. The number of clusters is the number of cylinders and the cluster center is the descriptive parameter of cylinders in the point cloud. Fig. 8 shows the result of using mean shift clustering for cylinder extraction for the block model.

## 5. Experimental results

### 5.1. Performance of the single cylinder fitting approach compared to other methods

To illustrate the robustness of the proposed method for fitting a single cylinder (Section 3) and estimating its descriptive parameters, the results were compared to those of three other methods: Gaussian sphere-based [6], Hierarchical clustering (HC) [1] and RANSAC-based methods [2]. Three different cylinders with synthetic, partial and missing data in Fig. 5 are used as datasets. The results are compared based on the following criterion:

- The difference between the estimated radius  $r$  and the ground truth radius  $\hat{r}$ :

$$MSE_r = \frac{1}{3} \sum_{i=1}^3 (\hat{r}_i - r_i)^2 \quad (17)$$

- The angle difference  $\vartheta$  (deg) between the estimated axis orientation and ground truth axis orientation:

$$MSE_\vartheta = \frac{1}{3} \sum_{i=1}^3 (\vartheta_i)^2 \quad (18)$$

- The distance is computed from the points to the detected cylinder:

$$MSE_e = \frac{1}{3} \sum_{i=1}^3 MSE_i \quad (19)$$

where  $MSE_i$  is computed with Eq. (6).

Table 1 lists the value of the criterion for these methods. The Gaussian sphere and Hierarchical clustering perform well but achieve less accurate results compared to the proposed method. As anticipated, the results obtained by RANSAC-based methods do not perform very well in the current example as indicated by the large MSE values. Our iterative approach helps to detect every point belonging to a given cylinder, and achieves a performance very close to ground truth. It illustrates that an iterative method based on distance and normal filtering works better than greedy single fitting for the Gaussian sphere and HC methods. It is worth mentioning that the results reported in Table 1 are averaged values over 10 runs for all the methods.

### 5.2. Multiple cylinder extraction

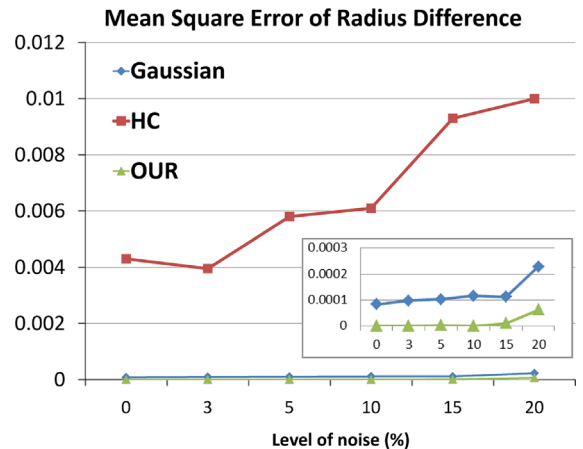
In this section, our algorithm (OUR) for multiple cylinder extraction is tested. The result is compared to the two other methods: Gaussian sphere-based (GAUSS) [6], Hierarchical clustering (HC) [1]. RANSAC-based method [2] is not done because it is hard to tune a set of parameters for complex models used in the experiments.

In a first experiment, a synthetic block model resampled to 12k points with three cylinders used for testing. The reason for resampling the data is to compare our method with the HC

**Table 1**

Comparison of the proposed method with Gaussian sphere, Hierarchical clustering (HC) and RANSAC based methods.

| Method       | $MSE_r$  | $MSE_\vartheta$ | $MSE_e$  |
|--------------|----------|-----------------|----------|
| RANSAC [2]   | 1.02E-02 | 0.01            | 1.08E-02 |
| HC [1]       | 4.56E-05 | 2.5E-04         | 7.6E-04  |
| Gaussian [6] | 5.87E-05 | 5.37E-03        | 6.5E-03  |
| Our method   | 4.06E-05 | 2.3E-04         | 5.56E-04 |



**Fig. 9.** MSE of detected radius with different levels of noise.

method which requires high computation time and memory. Then Gaussian noise with zero mean and standard deviations of 0–20% of the average distance between points was added to the point cloud data. Three methods were used to extract cylinders and estimate their parameters. The results are compared with ground-

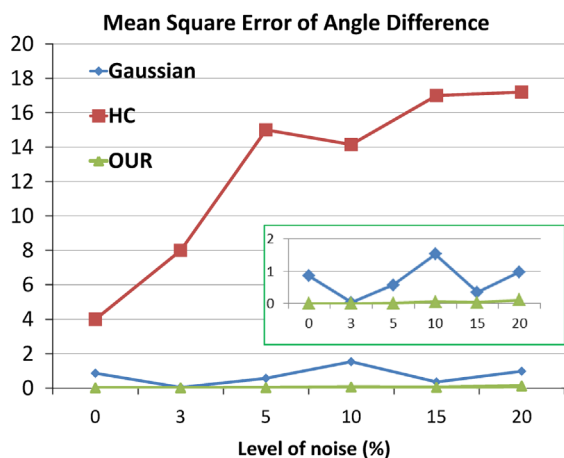


Fig. 10. MSE of detected axis direction with different levels of noise.

**Table 2**  
Threshold values used for this experiment and MSE for the three methods.

| Noise-outlier (%) | $\alpha$ | $\beta$ (rad) | $\epsilon$ | OUR      |          | GAUSS    |         | HC       |         |
|-------------------|----------|---------------|------------|----------|----------|----------|---------|----------|---------|
|                   |          |               |            | $MSE_r$  | $MSE_a$  | $MSE_r$  | $MSE_a$ | $MSE_r$  | $MSE_a$ |
| 0–0               | 50       | 0.95          | 0          | 1.01E–08 | 1.03E–04 | 3.68E–05 | 0.0622  | 4.43E–03 | 5.9203  |
| 5–5               | 50       | 0.9           | 1          | 5.22E–07 | 0.0280   | 5.68E–05 | 0.0959  | 5.10E–03 | 10.271  |
| 10–5              | 50       | 0.9           | 1          | 4.90E–06 | 0.0301   | 5.05E–04 | 0.2431  | 9.31E–03 | 13.069  |
| 15–10             | 50       | 0.85          | 2          | 1.62E–05 | 0.0304   | 7.94E–04 | 0.5857  | 6.94E–03 | 15.033  |
| 20–10             | 50       | 0.8           | 3          | 2.64E–05 | 0.0314   | 8.11E–03 | 0.8882  | 2.11E–02 | 18.062  |

truth values on the basis of the mean square error of radius difference  $MSE_r$  (Eq. (17)) and angle difference  $MSE_\theta$  (Eq. (18)). Observing Figs. 9 and 10, the HC method shows larger errors on detected radius and axis direction. The Gaussian sphere method achieves better results than HC but still results in averaged MSE 20 times larger than the proposed method.

We tested the proposed method, GAUSS method and HC method for different levels of noise and outliers for the resampled block model. First, Gaussian noise with zero mean and standard deviations of 0–20% of the average distance between points was added to the point cloud. In addition, 2% of the points were added as outliers with a magnitude 0–10% of the average distance between points. Fig. 12 shows the detected cylinders for the block model. Table 2 lists the set of threshold values used for the dataset and averaged MSE of the estimated radiuses and angle differences of the three cylinders. These results were averaged over 10 runs. Despite noise and the presence of outliers, our method is robust for the detection of cylinders and the estimation of the radius and axis direction compared to the two other methods. It demonstrates that the proposed method using two validation steps helps to reduce the effect of noise and outliers. The proposed method was tested on datasets captured on the lab by the Creaform GO! SCAN handheld 3D scanner. Fig. 11 shows the real scan data, which contains many outliers and holes. The method still achieves good

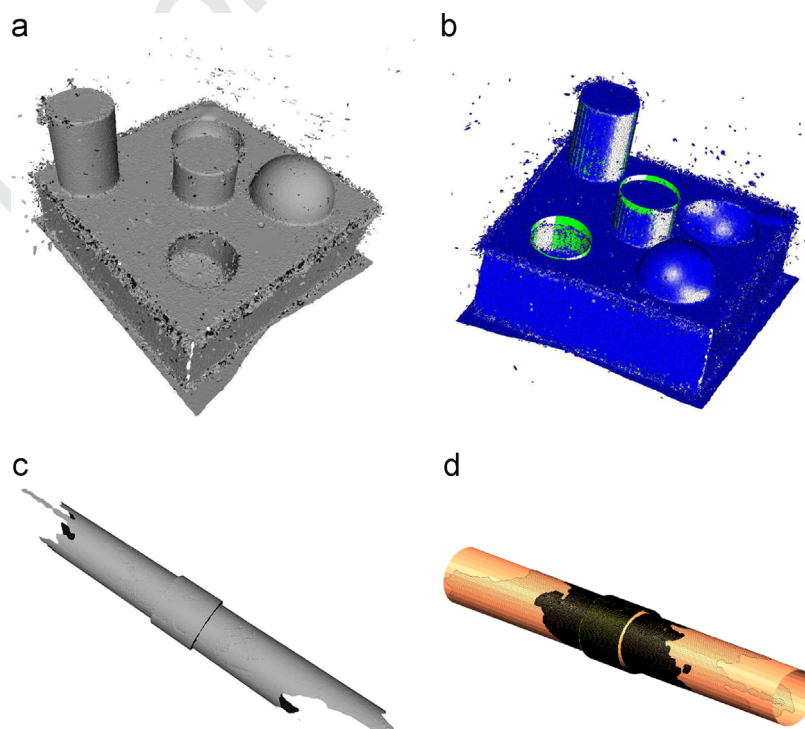
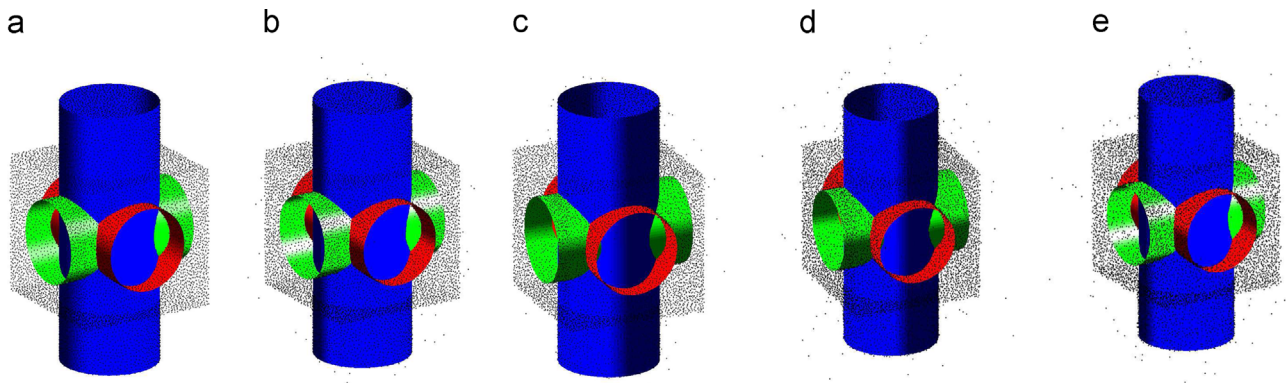
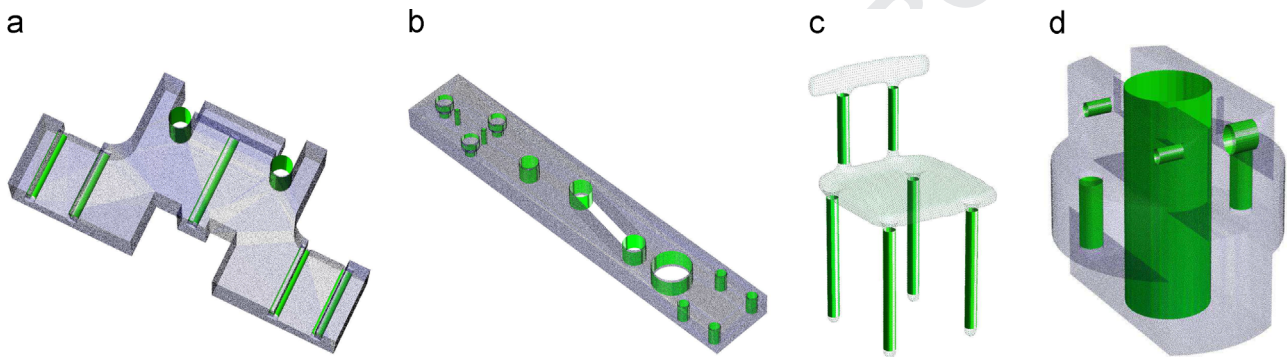


Fig. 11. Results on real scanned data captured by Creaform GO!SCAN 3D sensor (<http://www.goscan3d.com>) ( $\alpha = 50$ ;  $\beta = 0.9$ ;  $\epsilon = 2$ ). (a) Scan data 1. (b) Results of detected cylinders (green). (c) Scan data 2 of a pipeline. (d) Results of detected cylinders. Zoom in to see connector between pipes.



**Fig. 12.** Cylinders detected with different levels of noise and outliers for the block model. Table 2 lists the threshold values used in the experiment and MSE compared to other methods. (a) Data without noise and outliers. (b) 5% of noise and 5% outliers. (c) 10% of noise and 5% outliers. (d) 15% of noise and 10% outliers. (e) 20% of noise and 10% outliers.



**Fig. 13.** Cylinder extraction from point cloud of CAD models ( $\alpha = 50; \beta = 0.95; \epsilon = 0$ ). (a) Bracket. (b) Linkage arm. (c) Chair. (d) Socket.



**Fig. 14.** Results on real point cloud data: rolling stage [2], Data 1, Data 2 and Data 3 [14]. The first column shows the original models. Point clouds are shown in the second column. The third column shows the detected cylinders. Table 3 shows the number of detected cylinders. The thin and small cylinders are not detected ( $\alpha = 50; \beta = 0.85; \epsilon = 2$ ).

results without using any preprocessing step (e.g. denoising or hole-filling).

Fig. 13 shows multiple cylinders detected from CAD models. We have tested our method on real point cloud datasets used in the literature [2,14]. Fig. 14 shows the results of these experiments. These results show that our method is successful in extracting most of the complete and partial cylinders in point clouds of complex models.

### 5.3. Computation time

The performance of the method was evaluated in terms of computation time. It is compared to the Gaussian sphere-based method (GAUSS) [6] and Hierarchical clustering (HC) [1]. These methods were implemented in MATLAB on a 3.2 GHz Intel Core i7 platform. As mentioned earlier, the HC method has high complexity with respect to time and memory, so the test models (block and joint) were downsampled before running the experiments. Fig. 15 shows the computational time for three methods. The proposed method is slightly slower than the Gaussian sphere method, but

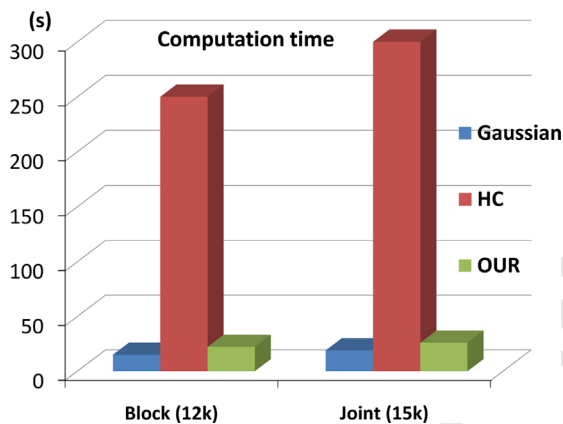


Fig. 15. Time comparison with other methods. Block and Joint model are resampled to 12k and 15k points respectively.

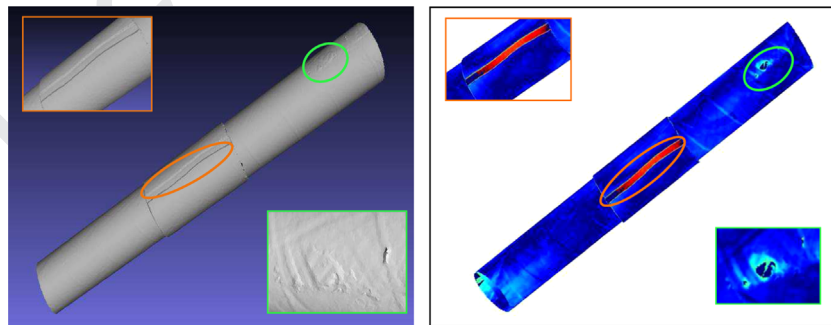


Fig. 16. Defects are detected from the pipe surface after extracting cylinders and their parameters.

Table 3

Computation time of our method for different models.

| Model name               | # Points | # Initial points | # Detected cylinders | Normal (s) | Curvature (s) | Fitting & Validation (min) |
|--------------------------|----------|------------------|----------------------|------------|---------------|----------------------------|
| Joint (Fig. 1)           | 50 000   | 2172             | 3                    | 3.52       | 12.12         | 2.79                       |
| Block (Fig. 8)           | 50 000   | 1971             | 3                    | 4.25       | 14.28         | 2.85                       |
| Rolling stage (Fig. 14)  | 50 000   | 3914             | 14                   | 2.36       | 10.16         | 3.41                       |
| Data 1 (Fig. 14)         | 105 802  | 7274             | 12                   | 10.38      | 24.89         | 7.21                       |
| Data 2 (Fig. 14)         | 841 30   | 2612             | 8                    | 6.01       | 21.88         | 3.57                       |
| Data 3 (Fig. 14)         | 141 267  | 3767             | 10                   | 7.62       | 27.45         | 4.32                       |
| Scan data 1 (Fig. 11(a)) | 289 393  | 7339             | 3                    | 12.95      | 30.74         | 15.71                      |
| Scan data 2 (Fig. 11(c)) | 181 504  | 7930             | 2                    | 22.97      | 50.52         | 10.4                       |

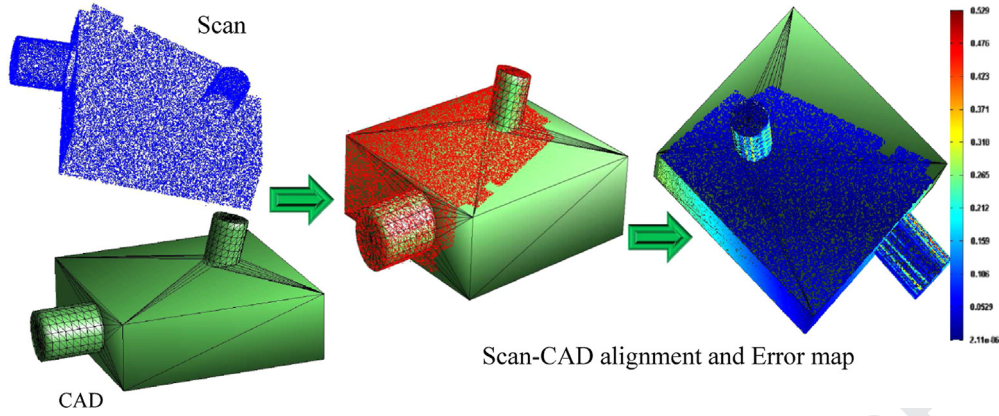
much faster than the HC method, because the proposed method merges multiple points in each iteration, but HC method only merges one point into cluster. Table 3 shows the computation time of the method averaged over 5 runs. The bottleneck of the algorithm is the fitting and validation steps. Computation time depends on how many initial points were extracted at the preprocessing step and the number of data points in the point cloud (see Table 3). Therefore, the processing time should drop if a resampling method is applied on the initial points. Furthermore, the whole process is implemented without parallel processing. We believe that a C++ implementation and parallel processing would speed up the process significantly.

### 5.4. Applications

The proposed algorithm could be used for various applications such as inspection, reverse engineering, segmentation, and registration. The first application is defect detection in Fig. 16. The surface of pipes often contains defects originating from many sources such as the manufacturing process or chemical oxidation. After extracting the cylinders and their parameters, some defect areas are detected easily by computing the distance from data points to the surface of the detected cylinder, as shown in Fig. 16.

Another interesting application for cylinder extraction is scan-cad registration. While other descriptors [28] failed to handle this problem, detected cylinders can be used as features to align scan-scan or scan-cad data [4]. Fig. 17 shows an example on scan-cad registration and quality control.

One more promising result of the proposed method is to extract pipelines from large datasets of industrial plants, as shown in Fig. 1. This kind of data contains significant noise and many outliers, as well as cylinders making adjoints. In the future we plan to implement preprocessing steps to split the point cloud into smaller datasets which would improve the performance of cylinder extraction that could then be merged into complete cylinders. These detected cylinders are input for pipeline reconstruction problems.



**Fig. 17.** Registration result for scanned data-CAD model using detected cylinders. Blue points, which are scanned data, are transformed to red points after alignment. The transformation is computed by using detected cylinders from both the scanned data and the CAD model [4].

## 6. Conclusion

We have proposed a novel algorithm for extracting cylinders and estimating their parameters from scanned 3D point cloud data. A novel validation method is also proposed to assign a confidence score to the detected cylinders. The algorithm is tested on various complex models and different levels of noise and outliers. The quality and robustness of the results suggests that the algorithm can be used in several applications such as reverse engineering and quality control, and registration.

The future work will be to apply our algorithm to handle large data from industrial plants or urban area. Moreover, this framework could also be extended to other types of primitives such as spheres, cones and toruses. We also plan to implement the algorithm in C++.

## Acknowledgments

The authors thank the AIM@SHAPE and the authors in [2,14] for making the models available. Thanks to my colleagues, Somayeh Hesabi for providing the scanned data in Fig. 11(c) and Fig. 16 and Annette Schwerdtfeger for proofreading the manuscript. We are grateful to Dr. Yong-Jin Liu for providing datasets in Fig. 1. This research project was supported by the NSERC/Creaform Industrial Research Chair on 3-D Scanning.

## Appendix A. Normal vector computation

Classical principal component analysis [20] is a popular method for normal vector estimation. However, point cloud density is non-uniform, a weight is assigned to each point in the neighborhood of a given point. This weight is derived from the distance of the point to the centroid. The covariance matrix  $CV$  of the neighborhood  $p_i = \{p_{ix}, p_{iy}, p_{iz}\}$  is computed as in Eq. (20) and the eigenvector corresponding to the smallest eigenvalue is considered as the normal vector  $n_i$  at point  $p_i$ .

$$CV_i = \sum_{j \in N(i)} \mu_j (p_j - \bar{p}_i)^T (p_j - \bar{p}_i) \in R^{3 \times 3} \quad (20)$$

where  $\bar{p}_i = (1/|N(i)|) \sum_{j \in N(i)} p_j$  is  $p_i$ 's local data centroid,  $N(i)$  is the neighborhood around the point  $p_i$  and  $|N(i)|$  is a set to 50 in our experiments.  $\mu_j$  is a Gaussian weight  $\mu_j = \exp(-(d_j^2/|N(i)|^2))$  where  $d_j = \|p_j - \bar{p}_i\|$ .

## Appendix B. Principal curvature computation

Taubin's method [22] is referred to as an efficient and reliable method for principal curvature estimation [23], which was improved by Hameiri et al. [24]. The main idea is summarized as follows. Let  $N(i)$  be the set of points in the neighborhood of point  $p_i$ .  $\vec{T}_{ij}$  is the tangent being a projection of the vector between  $p_i$  to  $p_j \in N(i)$  to the tangent plane at  $p_i$ .  $k_{ij}$  is the normal curvature associated with  $\vec{T}_{ij}$ , so  $M_{p_i}$  is defined as

$$M_{p_i} = \sum_{j \in N(i)} w_{ij} k_{ij} \vec{T}_{ij} \vec{T}_{ij}^T = [\vec{T}_1 \vec{T}_2 \vec{N}] \begin{pmatrix} e_1 & 0 & 0 \\ 0 & e_2 & 0 \\ 0 & 0 & 0 \end{pmatrix} [\vec{T}_1 \vec{T}_2 \vec{N}]^T \quad (21)$$

where  $k_{ij} \simeq 2\vec{n}_i^T (p_{ij} - p_i) / \|p_{ij} - p_i\|^2$  and  $w_{ij} \sim 1/\|p_{ij} - p_i\|$  is proportional to the inverse of the geometric distance from  $p_i$  to  $p_{ij}$  with  $\sum_{j \in N(i)} w_{ij} = 1$ .  $\vec{T}_1$  and  $\vec{T}_2$  denote the principal directions associated with the principal curvatures  $k_1$  and  $k_2$ , respectively.  $\vec{N}$  is the normal vector to the surface at a given point  $p_i$ . Assume that  $\hat{\theta}$  is the angle between  $\vec{T}$  and  $\vec{T}_1$ , and  $\theta_{ij}$  is the angle between  $\vec{T}_{ij}$  and  $\vec{T}$ , so

$$\begin{aligned} e_1 &= \vec{T}_1^T M_{p_i} \vec{T}_1 = \sum_{j \in N(i)} w_{ij} k_{ij} \vec{T}_1^T \vec{T}_{ij} \vec{T}_{ij}^T \vec{T}_1 \\ &= \sum_{j \in N(i)} w_{ij} [k_1 \cos^4(\theta_{ij} + \hat{\theta}) \\ &\quad + k_2 \sin^2(\theta_{ij} + \hat{\theta}) \cos^2(\theta_{ij} + \hat{\theta})] \\ &= Ak_1 + Bk_2 \end{aligned} \quad (22)$$

and

$$\begin{aligned} e_2 &= \vec{T}_2^T M_{p_i} \vec{T}_2 = \sum_{j \in N(i)} w_{ij} k_{ij} \vec{T}_2^T \vec{T}_{ij} \vec{T}_{ij}^T \vec{T}_2 \\ &= \sum_{j \in N(i)} w_{ij} [k_1 \sin^2(\theta_{ij} + \hat{\theta}) \cos^2(\theta_{ij} + \hat{\theta}) + k_2 \cos^4(\theta_{ij} + \hat{\theta})] \\ &= Bk_1 + Ck_2 \end{aligned} \quad (23)$$

where

$$\begin{aligned} A &= \sum_{j \in N(i)} w_{ij} \cos^4(\theta_{ij}); & B &= \sum_{j \in N(i)} w_{ij} \sin^2(\theta_{ij}) \cos^2(\theta_{ij}); \\ C &= \sum_{j \in N(i)} w_{ij} \sin^4(\theta_{ij}) \end{aligned} \quad (24)$$

The principal curvatures are computed by solving Eqs. (22) and (23) with  $A, B, C$  defined in Eq. (24). The reader is referred to [24] for more details.

## References

- [1] Attene M, Patanè G. Hierarchical structure recovery of point-sampled surfaces. *Comput Graph Forum* 2010;29(6):1905–20.
- [2] Schnabel R, Wahl R, Klein R. Efficient ransac for point-cloud shape detection. *Comput Graph Forum* 2007;26(2):214–26.
- [3] Varady T, Martin RR, Cox J. Reverse engineering of geometric models—an introduction. *Comput Aided Des* 1997;29(4):255–68.
- [4] Rabbani T, Dijkman S, van den Heuvel F, Vosselman G. An integrated approach for modelling and global registration of point clouds. *{ISPRS} J Photogramm Remote Sens* 2007;61(6):355–70.
- [5] Liu Y-J, Zhang J-B, Hou J-C, Ren J-C, Tang W-Q. Cylinder detection in large-scale point cloud of pipeline plant. *IEEE Trans Vis Comput Graph* 2013;19(10):1700–7.
- [6] Chaperon T, Goulette F. Extracting cylinders in full 3d data using a random sampling method and the gaussian image. In: *Proceedings of the vision modeling and visualization conference 2001*, 2001. p. 35–42.
- [7] Rabbani T, van den Heuvel F. Efficient hough transform for automatic detection of cylinders in point clouds. In: *ISPRS WG III/3, III/4, vol. 3*, 2005. p. 60–5.
- [8] Lozano-Perez T, Grimson W, White S. Finding cylinders in range data. In: *Proceedings of the IEEE international conference on robotics and automation*, vol. 4, 1987. p. 202–7.
- [9] Lukács G, Martin R, Marshall D. Faithful least-squares fitting of spheres, cylinders, cones and tori for reliable segmentation. In: *Proceedings of the 5th European conference on computer vision*. Freiburg, Germany: Springer; 1998. p. 671–86.
- [10] Benko P, Kós G, Várady T, Andor L, Martin R. Constrained fitting in reverse engineering. *Comput Aided Geom Des* 2002;19(3):173–205.
- [11] Taubin G. Estimation of planar curves, surfaces, and nonplanar space curves defined by implicit equations with applications to edge and range image segmentation. *IEEE Trans Pattern Anal Mach Intell* 1991;13(11):1115–38.
- [12] Attene M, Falcidieno B, Spagnuolo M. Hierarchical mesh segmentation based on fitting primitives. *Visual Comput* 2006;22(3):181–93.
- [13] Pratt V. Direct least-squares fitting of algebraic surfaces. *ACM SIGGRAPH Comput Graph* 1987;21(4):145–52.
- [14] Li Y, Wu X, Chrysathou Y, Sharf A, Cohen-Or D, Mitra NJ. Globfit: consistently fitting primitives by discovering global relations. *ACM Trans Graph* 2011;30(4):52:1–12.
- [15] Fischler MA, Bolles RC. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun ACM* 1981;24(6):381–95.
- [16] Bolles RC, Fischler MA. A ransac-based approach to model fitting and its application to finding cylinders in range data. In: *Proceedings of the 7th international joint conference on artificial intelligence, IJCAI'81*, vol. 2. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.; 1981. p. 637–43.
- [17] Duda RO, Hart PE. Use of the hough transformation to detect lines and curves in pictures. *Commun ACM* 1972;15(1):11–5.
- [18] Olson CF. Locating geometric primitives by pruning the parameter space. *Pattern Recognit* 2001;34(6):1247–56.
- [19] do Carmo MP. *Differential geometry of curves and surfaces*. Englewood Cliffs, NJ: Prentice-Hall; 1976.
- [20] Hoppe H, DeRose T, Duchamp T, McDonald J, Stuetzle W. Surface reconstruction from unorganized points. In: *Proceedings of the 19th annual conference on computer graphics and interactive techniques*, New York, USA, 1992. p. 71–8.
- [21] Jolliffe I. *Principal component analysis*. Springer, Verlag; 1986.
- [22] Taubin G. Estimating the tensor of curvature of a surface from a polyhedral approximation. In: *Proceedings of the fifth international conference on computer vision*. IEEE; 1995. p. 902–7.
- [23] Magid E, Soldea O, Rivlin E. A comparison of Gaussian and mean curvature estimation methods on triangular meshes of range image data. *Comput Vision Image Underst* 2007;107(3):139–59.
- [24] Hameiri E, Shimshoni I. Estimating the principal curvatures and the Darboux frame from real 3-d range data. *IEEE Trans Syst Man Cybern Part B: Cybern* 2003;33(4):626–37.
- [25] Otsu N. A threshold selection method from gray-level histograms. *IEEE Trans Syst Man Cybern* 1979;9:62–6.
- [26] Lamiroy B, Gaucher O, Fritz L, et al. Robust circle detection. In: *Proceeding of the 9th international conference on document analysis and recognition, ICDAR'07*, vol. 1, 2007. p. 526–30.
- [27] Comaniciu D, Meer P. Mean shift: a robust approach toward feature space analysis. *IEEE Trans Pattern Anal Mach Intell* 2002;24(5):603–19.
- [28] Salvi J, Matabosch C, Fofi D, Forest J. A review of recent range image registration methods with accuracy evaluation. *Image Vision Comput* 2007;25(5):578–96.